1.3.System Description

The Flight Booking System operates as a multi-tier application where the database acts as the core intelligence layer. Users interact with the system by inputting search parameters such as city pairs, travel dates, and preferred cabin classes. The system's logic engine then queries the database to return a filtered list of available flights, sorted by price or duration. A key feature of this system is the interactive seat map, which pulls real-time data to show which spots are occupied, restricted, or available for selection. Once a user picks a seat, the system initiates a "soft lock" or temporary reservation, preventing others from choosing it while the payment is being processed. This temporary state is governed by a timer; if the payment isn't confirmed within a set window, the seat is released back into the pool. Upon successful transaction through the integrated gateway, the status shifts to "confirmed," and an e-ticket is generated. The system also includes a comprehensive administrative backend where flight paths can be adjusted and pricing models updated dynamically. For passengers, a "Manage My Booking" portal allows for modifications, cancellations, and meal selections. This holistic approach ensures that every phase of the traveler's lifecycle is captured and managed within a single, unified digital environment.

1.4.Requirements

The functional requirements represent the core actions the system must perform to be useful to its end users. First, it must support a secure registration and profile management system where users can

store their travel preferences and loyalty points. Second, the search functionality must be highly granular, allowing for multi-city itineraries and flexible date searches. Third, the reservation engine must handle the logic of connecting flights, ensuring that layover times are sufficient and baggage is tracked. Fourth, the payment module must interface with various providers, including credit cards, digital wallets, and bank transfers. Fifth, the booking management system must automate the generation of receipts and boarding passes in various formats. On the other hand, non-functional requirements define the quality of the experience. High performance is required to prevent user abandonment during the checkout process. Scalability is essential so the infrastructure can grow alongside the airline's fleet. Security requirements mandate that the system be hardened against SQL injection and cross-site scripting attacks. Reliability ensures the system remains operational of the time, even during maintenance. Finally, the response time for the database must be optimized to ensure that the user interface feels snappy and intuitive, regardless of the complexity of the underlying search query.

1.5. Database Design

The architecture of this database is built upon a foundation of clearly defined entities that mirror real-world aviation components. At the heart of the design are the Passengers and Flights tables, which hold the fundamental data for every transaction. The Airlines table stores carrier-specific information, while the Seats table provides a granular map of every individual spot on every aircraft in the fleet. To link these, a Bookings table acts as a relational bridge, connecting a specific passenger to a specific seat on a specific flight. The Payments table is isolated to ensure that financial records can be audited independently of travel records. Each table is assigned a primary key, typically a unique UUID, to ensure that no two records can ever be confused.

Foreign keys are meticulously mapped to enforce "referential integrity," meaning you cannot have a booking for a flighhat each piece of information, such as a passenger's email, is stored in exactly one place.t that does not exist. We also implement "check constraints" to ensure that values like "Price" or "Seat Number" follow logical formats. To speed up the retrieval of data, we use indexes on frequently searched columns like Origin_City and Departure_Date. This design avoids data redundancy by ensuring t